

Keywords: KV cache • speculative eviction • efficient reasoning • LLM inference

Random Attention: Rethinking KV Cache Eviction for Efficient Reasoning

Scoring Cached Tokens Adds Nothing — Random Eviction with Prompt Protection Matches the Best Evictors at 32–43% Higher Throughput

By Heng Wang, Jieliu Qiu, Wenting Zhao, Cheng Qian, Liangwei Yang, Jiawei Han, Heng Ji, Silvio Savarese, Shelby Heinicke, Huan Wang (Salesforce AI Research; UIUC; Cornell; et al.)

arXiv:2609.03430 • EMNLP 2026

Long chain-of-thought reasoning turns the KV cache into a severe memory bottleneck. Every token in a reasoning trace adds a key-value pair; at thousands of tokens per query, the cache overflows batch capacity and degrades throughput. Every existing KV cache compression method responds with the same paradigm: score each cached token by some proxy for future importance, then keep the top scorers. Random Attention discards the scores entirely. It keeps the prompt tokens and evicts all others uniformly at random — and across four models and six reasoning tasks matches the best-performing scored evictors while achieving 32–43% higher throughput in vLLM.

AT A GLANCE

Problem: Long CoT reasoning makes the KV cache a memory bottleneck; existing eviction methods rely on expensive importance scoring.

Why it matters: Production inference systems serving reasoning models need maximal throughput; scoring overhead is non-trivial.

Method: Keep prompt KV cache; evict reasoning-trace tokens uniformly at random per attention head. No score computed.

Result: Matches strongest evictors on 6 tasks; 32–43% higher throughput at 50% cache budget in vLLM.

Evidence: Ablations isolating prompt protection from scoring; redundancy analysis across text and attention heads.

The Central Observation

Every KV cache evictor in the literature shares a core design assumption: the identity of which tokens to retain matters, and a good scoring function is needed to identify them. H2O uses cumulative attention weight; StreamingLLM uses recency with attention sinks; LESS

uses gradient-based estimates; SnapKV clusters similar keys.

Random Attention shows this assumption is false, at least for long reasoning traces. The scoring signal contributes almost nothing to the final task performance, once you control for whether the prompt is preserved.

Why the Scoring Signal Is Worthless

The authors run controlled ablation experiments across all four model families:

Condition A: Prior evictor, prompt not explicitly protected — the prompt may happen to be evicted.

Condition B: Prior evictor with explicit prompt protection.

Condition C: Random eviction, no prompt protection.

Condition D (Random Attention): Random eviction with prompt protection.

The results show Condition B $\approx$ Condition D $\gg$ Conditions A and C. The gap between A and D is almost entirely explained by whether the prompt is preserved, not by whether scoring is used. The gap between B and D (evictor scoring vs. random after prompt protection) is negligible.

Two Mechanisms Explaining Redundancy

Why does random eviction of the reasoning trace lose almost nothing?

Text-level redundancy. Models engaged in chain-of-thought reasoning restate the facts they need as they work. If the model needs P to derive Q three steps later, it typically restates P (or a consequence of P) at the derivation site. Evicting the original occurrence of P loses nothing because the restatement survives. The authors measure text-level redundancy: roughly 60% of trace tokens can be deleted while preserving QA-measured semantic content.

Head-level redundancy. Each attention head maintains its own copy of the KV cache. Information encoded at a given token is distributed across heads. When eviction is performed independently per head (as in Random Attention), the probability that a fact encoded in m heads is retained by at least one head is:

$$P(\text{fact retained}) = 1 - (1 - p)^m$$

For $m = 6$ heads and $p = 0.5$ budget: $P = 1 - 0.5^6 = 0.984$. This near-certainty of retention means no scored selection is needed.

Technical Formulation

Let $\mathcal{P}$ be the set of prompt token positions and $\mathcal{T}$ the set of reasoning-trace positions. Given a cache budget B ,

standard evictors compute:

$$\mathcal{R} = \mathcal{P} \cup \text{top}_k^{\text{score}}(\mathcal{T}), \quad k = B - |\mathcal{P}|$$

Random Attention replaces scored selection with uniform sampling:

$$\mathcal{R}^{(h)} = \mathcal{P} \cup \text{sample}_k^{\text{uniform}}(\mathcal{T})$$

performed independently per attention head h . No score function is defined or computed.

Experimental Findings

Experiments span four models — DeepSeek-R1-Distill-Qwen-7B, DeepSeek-R1-Distill-Qwen-14B, QwQ-32B, and DeepSeek-R1-32B — and six tasks: AIME 2024, AIME 2025, AMC, MATH500, LiveCodeBench, and GPQA Diamond. At 50% cache budget:

Method	AIME 2024	MATH500
Full KV cache	72.3%	91.4%
H2O (50%)	68.1%	89.3%
StreamingLLM (50%)	67.4%	88.7%
Random Att. (50%)	68.4%	89.5%

Random Attention matches or slightly exceeds scored evictors, while serving 32–43% higher throughput in vLLM deployment.

Throughput Analysis

The throughput advantage has two sources. First, eliminating the score computation removes a matrix product over the full KV sequence per generation step, saving roughly 15% of the per-step compute at 50% budget. Second, uniform random eviction produces better memory access patterns than score-sorted eviction (which leaves a non-contiguous, score-ranked subset in the cache), saving the remaining 17–28%.

Ablations and Interpretation

Per-head vs. global eviction: Per-head independent sampling outperforms global uniform eviction by 1.2 AIME points, confirming that head-level redundancy requires per-head independent treatment.

Tight budgets (25%): Random Attention is more robust than scored evictors at 25% budget, where its MATH500 accuracy matches scored evictors at 50%.

Task type: Temporal and procedural tasks (LiveCodeBench) benefit slightly more from Random Attention than purely mathematical tasks, consistent with higher restatement frequency in procedural traces.

Prompt fragility confirmed: If the prompt is evicted even partially (Condition C), accuracy drops 17+ points

on AIME 2024 — confirming that the prompt, not the trace, is semantically irreplaceable.

Reference

Heng Wang et al. **Random Attention: Rethinking KV Cache Eviction for Efficient Reasoning.** arXiv:2609.03430, EMNLP 2026. <https://arxiv.org/abs/2609.03430>

Keywords: multimodal LLM • hallucination • visual grounding • object hallucination

Beyond Language Priors: Diagnosing and Fixing Visual-Origin Hallucinations in Multimodal LLMs

Cosine Similarity and Grad-CAM Entropy Reveal That Half of Object Hallucinations Originate in the Visual Encoder, Not the Language Model

By Peiyang Xu, Xiaopei Zhu, Jun Zhu, Xiaolin Hu (Tsinghua University)

arXiv:2609.00231 • Preprint, August 2026

Object hallucination in multimodal large language models (MLLMs) — the generation of text describing objects not present in the image — is a persistent failure mode that undermines trust in deployed systems. The field has converged on a near-consensus explanation: the language model component over-relies on learned co-occurrence statistics (language priors), producing objects that are likely in the described scene even when they are absent from the image. This paper challenges that consensus. Using cosine similarity analysis and Smooth Grad-CAM entropy, the authors show that 38–52% of object hallucinations across three MLLM architectures originate not in the language prior but in failures of the visual encoder itself.

AT A GLANCE

Problem: Object hallucination in MLLMs is blamed on language priors; visual encoding failures are ignored.

Why it matters: Misdiagnosing the root cause leads to interventions (decoding fixes) that do not address visual failures.

Method: Cosine similarity and Smooth Grad-CAM entropy to classify hallucinations as visual-origin vs. language-prior-origin.

Result: 38–52% of hallucinations are visual-origin; targeted visual fixes reduce POPE hallucination rate by 4.1 points and MSCOCO-Hallu rate by 37% relative.

Evidence: Three MLLM families; POPE, MME, MSCOCO-Hallu benchmarks; ablations of visual contrastive training and attention focusing.

The Language Prior Hypothesis and Its Limits

The dominant account of object hallucination posits that MLLMs inherit strong language priors from pre-training: the language model has seen “refrigerator” co-occurring

with “kitchen” thousands of times and will generate it even when the image shows an empty kitchen. Prior work has proposed remedies at the language side: instruction tuning with hallucination-negative examples, post-hoc decoding corrections, and contrastive decoding between image-conditioned and image-unconditioned outputs.

These language-side fixes improve hallucination metrics, but do not fully close the gap with human performance. This paper asks whether there is a second root cause that language-side fixes cannot address.

Diagnosing Visual-Origin Hallucination

The authors propose two diagnostic metrics that distinguish visual-origin from language-prior-origin hallucination.

Image-text cosine similarity measures how well the visual embedding aligns with the text of the generated output:

$$\text{sim}(v, t) = \frac{\mathbf{f}_v \cdot \mathbf{f}_t}{\|\mathbf{f}_v\| \|\mathbf{f}_t\|}$$

If the visual encoder correctly represents the image content, $\text{sim}(v, t)$ should be high for faithful outputs. A low $\text{sim}(v, t)$ for a hallucinatory output indicates that the generated text does not correspond to the visual representation — suggesting the visual encoder failed.

Smooth Grad-CAM entropy measures the diffuseness of image regions attended during generation:

$$H_{\text{Grad-CAM}} = - \sum_r p_r \log p_r$$

where p_r is the normalized Smooth Grad-CAM activation for image region r . High entropy means the model fails to focus on task-relevant objects. For faithful outputs, entropy is low; for visual-origin hallucinations, entropy is systematically elevated.

Using these two metrics as a joint classifier, the authors assign each hallucinated object instance to one of three categories:

- **Language-prior-origin:** high sim, low entropy, object absent from image.
- **Visual-origin:** low sim or high entropy, regardless of whether the object could be predicted from language priors.
- **Ambiguous:** both factors elevated.

Quantitative Decomposition

Across three architectures (LLaVA-1.5, InstructBLIP, MiniGPT-4) on MSCOCO-Hallu, the decomposition results are:

Model	Language	Visual	Ambig.
LLaVA-1.5	41%	44%	15%
InstructBLIP	38%	47%	15%
MiniGPT-4	45%	40%	15%
Average	41%	44%	15%

The visual-origin fraction is consistently comparable to or larger than the language-prior fraction, establishing it as a primary, not secondary, cause.

Targeted Remediation

The authors propose two complementary fixes targeting the visual encoder.

Visual contrastive training pushes image embeddings toward representations of faithful captions and away from unfaithful ones:

$$\mathcal{L}_{\text{contrast}} = -\log \frac{\exp(\text{sim}(v, t^+)/\tau)}{\exp(\text{sim}(v, t^+)/\tau) + \sum_j \exp(\text{sim}(v, t_j^-)/\tau)}$$

where t^+ is the ground-truth caption and t_j^- are negative captions (hallucination-containing descriptions generated by the model).

Attention focusing regularization reduces visual entropy during training by penalizing diffuse visual attention:

$$\mathcal{L}_{\text{focus}} = \lambda_2 \cdot H_{\text{Grad-CAM}}$$

The full training objective combines both:

$$\mathcal{L} = \mathcal{L}_{\text{LM}} + \lambda_1 \mathcal{L}_{\text{contrast}} + \lambda_2 \mathcal{L}_{\text{focus}}$$

Experimental Results

Model	POPE Acc.	Hallu. Rate
LLaVA-1.5 (baseline)	85.3%	28.4%
+ Language-side fix (ICD)	87.1%	24.9%
+ Visual contrastive training	88.6%	20.1%
+ Full method	89.4%	17.8%

The full method reduces hallucination rate by 37.3% relative on MSCOCO-Hallu and improves POPE accuracy by 4.1 points, without degrading object recognition or VQA performance.

Ablations and Interpretation

Contrastive vs. focus ablation: Contrastive training accounts for 65% of the hallucination reduction; attention focusing contributes the remaining 35%.

Cross-architecture AUC: The diagnostic metrics predict visual-origin hallucinations with AUC 0.78–0.83 across all three families, confirming the finding generalises beyond a single model.

No regression on downstream tasks: VQAv2 and ScienceQA scores are unchanged, confirming the intervention is targeted and does not erode general multimodal capability.

Diagnostic threshold sensitivity: The 41–44% visual-origin estimate shifts by at most ± 6 points across threshold choices, showing the result is robust to the classification boundary.

Reference

Peiyang Xu, Xiaopei Zhu, Jun Zhu, and Xiaolin Hu. **Beyond Language Priors: Diagnosing and Fixing Visual-Origin Hallucinations in Multimodal LLM.** arXiv:2609.00231, August 2026. <https://arxiv.org/abs/2609.00231>

Keywords: vision-language models • causal intervention • cross-modal attention • video understanding

From Vision to Language: Investigating Causal Information Flow in Multimodal Decision-Making

Causal Patching of Video-Text Attention Reveals That VLMs Ground Vision at Answer Tokens, with Nouns as Anchors and Temporal Reasoning Fragile

By Davide Testa, Hugh Mee Wong, Alessandro Lenci, Bernardo Magnini, Albert Gatt (FBK; Univ. Malta; Univ. Pisa; et al.)

arXiv:2609.05149 • Preprint, September 2026

Vision-language models (VLMs) are widely deployed for video understanding tasks — question answering, visual reasoning, action recognition. Yet the internal mechanism by which visual information is integrated into language-based decisions remains largely opaque. This paper applies layer-wise causal intervention on video-text attention pathways to answer a basic question: at which layer, and at which token positions, does visual information causally determine the model’s final answer? The answer is consistent across five VLMs spanning two orders of magnitude in parameters: visual information is integrated primarily when the model processes the candidate *answer* tokens, not the question. Nouns act as semantic anchors for cross-modal grounding; verbs carry temporal relational load; and temporal reasoning exhibits a distinct fragility suggesting models rely on linguistic cues rather than genuine frame-by-frame sequential understanding.

AT A GLANCE

Problem: Where and when do VLMs integrate visual signals when answering video questions?

Why it matters: Understanding integration sites guides architecture choices and reveals failure modes.

Method: Layer-wise causal patching of video-text attention activations; tested on spatial, causal, and temporal QA.

Result: Visual integration peaks at answer tokens; nouns anchor cross-modal binding; temporal reasoning fragility TFI=0.34.

Evidence: 5 VLMs (7B–72B); consistent across spatial, causal, and temporal tasks.

The Question

VLMs process videos as sequences of frame tokens and answer multiple-choice questions about their content. The prevailing assumption is that visual grounding is distributed throughout the forward pass. But mechanistic interpretability work on image-text models has suggested that cross-modal integration is often concentrated at specific layers and token positions. For video, with its additional temporal structure, the picture is even less clear.

This paper runs the first systematic causal intervention study on the cross-modal attention pathways of video VLMs, targeting three reasoning types: spatial (object location, spatial relations), causal (action outcomes, event causation), and temporal (sequence, duration, before/after).

Causal Patching Methodology

The central method is **causal activation patching** applied to cross-modal attention activations.

For a clean run on a video-question pair, all attention activations $\mathbf{A}_t^{(l)}$ at layer l , token position t are recorded. A corrupted run processes a corrupted video (shuffled or masked frames) to obtain degraded activations.

Causal patching restores the clean activation at a single (l^*, t^*) while all other activations remain corrupted:

$$\mathbf{A}_t^{(l)} = \begin{cases} \mathbf{A}_t^{(l)}[\text{clean}] & l = l^*, t = t^* \\ \mathbf{A}_t^{(l)}[\text{corrupted}] & \text{otherwise} \end{cases}$$

The **causal effect** of visual information at (l^*, t^*) is:

$$\text{CE}(l^*, t^*) = P(\text{correct} \mid \text{patched at } l^*, t^*) - P(\text{correct} \mid \text{fully corrupted})$$

High CE means visual information at that position is causally necessary.

Normalising by the maximum recoverable effect:

$$\widehat{\text{CE}}(l^*, t^*) = \frac{\text{CE}(l^*, t^*)}{P(\text{correct} \mid \text{clean}) - P(\text{correct} \mid \text{corrupted})}$$

Where Visual Integration Happens

The results are striking in their consistency. Across all five models and all three task types, the highest causal effects appear at the positions of *candidate answer tokens* — not the question, not the reasoning prefix. The model reads the video primarily when it evaluates each answer option against the video content.

	Spatial	Causal	Temporal
Peak CE at answer tokens	0.71	0.68	0.53
Peak CE at question tokens	0.18	0.22	0.19

Visual integration peaks at layers 60–75% of model depth, in the middle-to-late transformer layers.

Nouns Anchor, Verbs Carry Temporal Load

Aggregating causal effects by token part-of-speech reveals the role of different word classes:

$$\widehat{\text{CE}}_{\text{POS}}(\text{pos}) = \frac{1}{|T_{\text{pos}}|} \sum_{t \in T_{\text{pos}}} \sum_l \widehat{\text{CE}}(l, t)$$

POS	Spatial	Causal	Temporal
Noun share	58%	51%	44%
Verb share	12%	24%	41%
Other share	30%	25%	15%

Nouns dominate as binding sites for object identity and spatial information. Verbs grow in importance for causal and temporal tasks, where relational and sequential information must be extracted from the video.

Temporal Fragility

The **Temporal Fragility Index** (TFI) measures how much harder it is to recover visual information for temporal questions:

$$\text{TFI} = \frac{\widehat{\text{CE}}_{\text{spatial}} - \widehat{\text{CE}}_{\text{temporal}}}{\widehat{\text{CE}}_{\text{spatial}}}$$

TFI = 0.34 averaged across models. Restoring a clean visual signal for a temporal question recovers 34% less accuracy than for a spatial question, despite identical video content.

Combined with the high verb CE share on temporal tasks, this pattern suggests VLMs use linguistic temporal markers (verb tense, temporal adverbs, “before/after” phrases) rather than reconstructing sequential information across frames. Temporal expressions in answer candidates trigger reliance on linguistic temporal priors.

Experimental Details

Models: LLaVA-Video-7B, LLaVA-Video-72B, Qwen2.5-VL-7B, Qwen2.5-VL-72B, InternVL3-8B.

Video benchmarks: Spatial, causal, and temporal questions derived from standardised evaluation datasets; 4 answer candidates per question; balanced correct/incorrect.

Corruption: Frame shuffling and frame masking produce similar CE patterns, confirming that ordering information (not just content presence) drives temporal fragility.

Model scale: Larger models achieve higher absolute accuracy but the same qualitative pattern, suggesting these are architectural properties rather than capacity limitations.

Reference

Davide Testa, Hugh Mee Wong, Alessandro Lenci, Bernardo Magnini, and Albert Gatt. **From Vision to Language: Investigating Causal Information Flow in Multimodal Decision-Making.** arXiv:2609.05149, September 2026. <https://arxiv.org/abs/2609.05149>

Keywords: program synthesis • neural functions • adapter fine-tuning • natural language specifications

Compile by Training: Turning Natural-Language Specifications into Local Neural Functions

Teacher LLMs Generate Task-Specific Training Data at Compile Time; a Lightweight Adapter Runs Offline at 83.6% Semantic Accuracy Where Fast Compilers Achieve Zero

By Yuntian Deng, Pengyu Nie, Stuart Shieber (Harvard University; et al.)

arXiv:2609.04199 • EMNLP 2026 System Demonstrations

A natural-language specification — “given an address string, return the city component, normalised to title case, with abbreviations expanded” — describes a deterministic function. Current practice runs a large frontier LLM at inference time for every query, incurring repeated cost, requiring internet connectivity, and providing no persistent, versioned artifact. Compile by Training (CbT) converts the specification into a lightweight adapter that runs locally on a compact interpreter model, with no runtime teacher access. At compile time, teacher LLMs generate diverse input-output examples that train the adapter to implement the specification exactly. The result runs at 12 ms per query with 83.6% semantic accuracy on FuzzyBench-Hard — a benchmark where the Program-as-Weights fast compiler achieves zero exact matches.

AT A GLANCE

Problem: Repeating a frontier LLM call for every invocation of a fixed natural-language-specified task is expensive, slow, and requires cloud connectivity.

Why it matters: Many deployed NLP tasks are static, reused repeatedly, and need local, versioned, composable execution.

Method: Teacher LLMs enumerate examples; compact adapter is trained on them; runs offline at inference.

Result: 83.6% semantic accuracy on FuzzyBench-Hard (PAW: 0% exact); 12 ms/query local runtime; composes and versions like code.

Evidence: FuzzyBench-Hard, WebTask, AvatarControl; ablations on committee size, training set size, LoRA rank.

Motivation: From Repeated Inference to Compiled Functions

For a task that will be called millions of times — normalising product names, extracting structured fields from documents, controlling a 3D avatar from natural language — the cost of a frontier LLM call per invocation is prohibitive. The output distribution of the task is fixed by its specification; there is no need to re-solve it at runtime.

The Program-as-Weights (PAW) framework already embeds this intuition: a specification is treated as a “program” compiled into a neural network’s weights. PAW’s fast compiler generates examples in seconds and fine-tunes a small adapter. But fast compilation is shallow — too few examples, too little training — and fails on hard specifications.

CbT extends PAW with a deeper, more expensive compilation that produces far higher accuracy while preserving the local, fast, offline runtime.

Compile by Training: The Four Steps

Step 1: Example generation by teacher committee. Given specification s , a committee of K teacher LLMs is queried to generate diverse (x_i, y_i^*) input-output pairs. Multiple teachers reduce individual model errors.

Step 2: Validation by cross-reference. An input x is accepted with output y^* if at least k of K teachers agree on y^* . Disagreed pairs are discarded. The disagreement rate d serves as a specification ambiguity signal.

Step 3: Adapter training on curated examples. A LoRA adapter $\delta\theta$ for a compact local interpreter is trained on the validated dataset $\mathcal{D}_s = \{(x_i, y_i^*)\}$:

$$\mathcal{L}_{\text{CbT}} = - \sum_{(x, y^*) \in \mathcal{D}_s} \log p_{\theta + \delta\theta}(y^* | s, x)$$

Step 4: Local deployment. The compiled artifact $(s, \delta\theta)$ is stored. At inference time, the compact model with its adapter processes new inputs locally, without any teacher or internet access, at 12 ms per query.

Semantic Accuracy Metric

Standard exact-match fails for tasks with multiple valid output forms. CbT uses **semantic accuracy**:

$$\text{SemAcc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{sem-equiv}(f_\theta(x_i), y_i^*)]$$

where sem-equiv judges equivalence using a judge LLM, accepting different surface forms of the correct answer.

Accuracy vs. Compile Compute

Empirically, accuracy saturates as a function of training set size:

$$\text{SemAcc}(N_{\text{ex}}) \approx A - B \cdot e^{-\alpha N_{\text{ex}}}$$

with $A \approx 0.85$, $B \approx 0.84$, $\alpha \approx 0.003$ on FuzzyBench-Hard. Saturation occurs around $N_{\text{ex}} = 1000$ examples, corresponding to a compile time of approximately one minute (GPT-4o teacher, parallel calls).

Composition

CbT functions compose by chaining. If f_A implements specification s_A and f_B implements s_B , a composed function $f_{A \circ B}$ is compiled by: (1) generating a composed specification $s_{A \circ B}$; (2) using f_A to produce intermediate inputs for f_B as training examples. The resulting adapter implements $s_{A \circ B}$ locally without the intermediate function at inference time.

Experimental Results

Method	SemAcc	Compile	Runtime
Teacher (GPT-4o) runtime	91.2%	0 s	2.1 s/q
PAW fast compiler	51.3%	8 s	12 ms/q
CbT (ours)	83.6%	60 s	12 ms/q
CbT + validation	87.1%	120 s	12 ms/q

CbT closes 80% of the quality gap between PAW and teacher-at-runtime while matching PAW’s fast local inference. On WebTask, two-function compositions achieve 76.4% task success, confirming composition viability.

Ablations

Committee size: 3 teachers reduces disagreement from 31% (1 teacher) to 12% and improves SemAcc from 79.4% to 83.6%.

Training set size: 1000 examples saturates; 500 achieve 80.1%; 2000 add only 0.8 points.

LoRA rank: Rank 16 achieves full accuracy; rank 4 loses 2.1 points; rank 64 adds nothing.

Specification clarity: Disagreement rate $d > 0.4$ predicts low SemAcc; prompting users to clarify ambiguous specifications recovers 4.3 SemAcc points on average.

Practical applications: Website form-filling helpers and language-controlled 3D avatar motion demos confirm that CbT applies to realistic end-to-end tasks beyond benchmarks.

Reference

Yuntian Deng, Pengyu Nie, and Stuart Shieber. **Compile by Training: Turning Natural-Language Specifica-**

tions into Local Neural Functions. arXiv:2609.04199, EMNLP 2026 System Demonstrations. <https://arxiv.org/abs/2609.04199>

Keywords: diffusion transformer • image generation • open training recipe • Muon optimizer

LLaDA-Image: Building Strong Image Generators with Fully Open Training Recipes

A 6B Diffusion Transformer with Staged Training, Parameter-Free RMSNorm, and the Muon Optimizer Achieves State-of-the-Art on Qwen-Image-Bench with Full Open Release

By Chuyan Chen, Haoxing Chen, et al. (inclusionAI, Ant Group)

arXiv:2609.03796 • Preprint, September 2026

High-quality text-to-image generation has long been dominated by closed proprietary systems. LLaDA-Image breaks this pattern: a 6B Diffusion Transformer (DiT) trained from scratch within a unified framework alongside a frozen vision-language understanding module, achieving state-of-the-art scores of 53.53 (English) and 53.38 (Chinese) on Qwen-Image-Bench, with all model weights, training code, and stage-by-stage recipes publicly released. The key engineering decisions — staged training (image-only first, then paired text-image), parameter-free RMSNorm, and the Muon optimizer — are each responsible for significant quality and stability gains and are now fully reproducible by the community.

AT A GLANCE

Problem: Competitive text-to-image generation requires proprietary recipes, closed weights, and unreported training decisions.

Why it matters: Without reproducible open baselines, the community cannot build on, audit, or improve SOTA image generators.

Method: 6B DiT + frozen LLaDA2.0-Mini backbone; image-only pre-training then paired text-image training; parameter-free RMSNorm; Muon optimizer.

Result: SOTA Qwen-Image-Bench 53.53 EN / 53.38 ZH; Muon reduces training compute by 20%; Turbo variant in 2–4 steps.

Evidence: Qwen-Image-Bench, GenEval, T2I-CompBench; ablations on staged training, RMSNorm, and optimizer.

Architecture: Unified DiT Plus Frozen LLM

LLaDA-Image pairs two modules:

Image Generation Module (DiT). A 6-billion-parameter Diffusion Transformer trained from scratch to perform masked discrete diffusion over image tokens. The DiT models the conditional distribution $p(x_{\text{image}}|c_{\text{text}})$ where c_{text} is the conditioning signal from the understanding module.

Vision-Language Understanding Module. A frozen LLaDA2.0-Mini diffusion language model, which processes text prompts and provides conditioning for the DiT. Because LLaDA2.0-Mini was trained on both English and Chinese text, the system natively understands Chinese generation instructions — a key advantage over English-dominant baselines.

The two modules share an image tokeniser (VQ-VAE) that discretises images into sequences of tokens, enabling a unified token vocabulary.

Key Engineering Decision 1: Staged Training

Training starts with image-only pre-training before any text-image pair is introduced.

Stage 1 — Image-only pre-training (98M samples): The DiT is trained on unlabeled images using the masked diffusion objective with no conditioning signal. This establishes a strong visual generative prior: the model learns the manifold of natural images before it is asked to condition on language.

Stage 2 — Paired text-image mid-training (122M samples): The DiT is fine-tuned on (text, image) pairs with conditioning from the understanding module. Starting from a strong visual prior produces significantly better text-following than training on pairs from scratch.

Stage 3 — Instruction fine-tuning: A smaller high-quality stage sharpens precision on detailed captions and editing instructions.

The ablation is decisive: removing Stage 1 and starting with paired training reduces Qwen-Image-Bench score by 3.8 points.

Key Engineering Decision 2: Parameter-Free RMSNorm

Standard affine RMSNorm applies learned scale γ and bias β :

$$\text{RMSNorm}_{\text{affine}}(x) = \gamma \cdot \frac{x}{\|x\|_2} + \beta$$

At 6B parameters, the learned scale can drift during training, causing loss spikes that require manual intervention.

Parameter-free RMSNorm simply normalises to unit norm:

$$\text{RMSNorm}_{\text{free}}(x) = \frac{x}{\|x\|_2}$$

Fixing the output norm eliminates the scale drift channel, enabling stable training throughout. Applied uniformly

throughout the DiT, this removes approximately 12M parameters while improving training stability.

Key Engineering Decision 3: Muon Optimizer

Standard Adam-family optimisers update parameters along the raw gradient direction, which can inflate or collapse parameter norms. The Muon optimizer applies orthogonalised gradient updates:

$$\mathbf{G}_t^\perp = \text{NewtonSchulz5}(\mathbf{M}_t)$$

$$\theta_{t+1} = \theta_t - \eta \mathbf{G}_t^\perp$$

where $\mathbf{M}_t$ is the momentum-corrected gradient and NewtonSchulz5 applies five iterations of the Newton-Schulz iteration to orthogonalise $\mathbf{M}_t$. Orthogonal updates preserve parameter norms, improving both convergence speed and final quality.

Masked Diffusion Objective

For an image token sequence $x = (x_1, \dots, x_T)$, the DiT is trained to predict masked tokens:

$$\mathcal{L}_{\text{DiT}} = -\mathbb{E}_{t, \mathbf{m}} \left[\sum_{i: m_i=1} \log p_\theta(x_i | x_{\setminus m}, c, t) \right]$$

where $\mathbf{m}$ is a random mask, $x_{\setminus m}$ are unmasked tokens, c is text conditioning, and t is the diffusion timestep.

LLaDA-Image-Turbo: Distillation to 2–4 Steps

LLaDA-Image-Turbo is distilled from the full model using consistency distillation:

$$\mathcal{L}_{\text{turbo}} = \mathbb{E}_{x, t, s} \|f_\theta(x_t, t) - f_{\theta^-}(x_s, s)\|^2$$

where $t > s$ are timesteps on the same trajectory, f_θ is the student, and f_{θ^-} is an EMA teacher. Enforcing consistency along generation trajectories enables convergence in 2–4 steps at 94–97% of full model quality.

Experimental Results

Model	QB-EN	QB-ZH	GenEval
DALL-E 3	50.1	48.7	0.61
SD 3.5-L	49.6	—	0.71
Flux-1.1-pro	51.4	—	0.72
LLaDA-Image	53.53	53.38	0.74
LLaDA-Turbo (4-step)	51.8	51.4	0.71

Training compute: Muon reduces GPU-hours to convergence by 20% (75,500 vs. 94,400 with AdamW) and reaches the same validation loss 25% fewer steps.

Ablations

Stage 1 removal: −3.8 Qwen-Image-Bench points.

Affine vs. free RMSNorm: Affine causes training instability at 4B+ scale; free RMSNorm eliminates all spikes.

Muon vs. AdamW: Muon is +1.1 Qwen-Image-Bench points at the same compute budget.

Turbo step count: 4 steps retains 97% of quality; 2 steps 94%, at 8× speedup.

Chinese prompt following: The bilingual LLaDA2.0-Mini backbone provides SOTA Chinese scores where English-dominant baselines degrade significantly.

Reference

Chuyan Chen et al. (inclusionAI). **LLaDA-Image: Building Strong Image Generators with Fully Open Training Recipes**. arXiv:2609.03796, September 2026. <https://arxiv.org/abs/2609.03796>